PROCESSO SELETIVO • CICLO 2026/2027 • EDITAL OFICIAL

Programa de Residência Tecnológica em Sistemas Android Avançados

Desafio Técnico de Admissão, Nivelamento e Rastreabilidade Experimental

Projeto Alvo: *Android Sentinel Core • Telemetria, IA On-Device e Otimização Energética*

CARGA CURRICULAR	MODALIDADE DE AVALIAÇÃO	REGIME DE EXECUÇÃO	PONTUAÇÃO TOTAL
285h (210h Núcleo + 75h Trilha)	Código Aberto + Relatório Técnico	Individual • Autônomo	100 Pontos (Classificatório)

1. Apresentação do Programa e Alinhamento Pedagógico

O **Programa de Residência Tecnológica em Sistemas Android Avançados** é uma imersão técnico-científica voltada para a formação de engenheiros e pesquisadores capazes de atuar no estado da arte do sistema operacional Android e do Android Open Source Project (AOSP). O currículo compõe-se de um **Núcleo Comum de 210h** somado a **75h de Formação Especializada** em uma de três trilhas tecnológicas.

O objetivo deste desafio técnico é avaliar competências que abrangem desde a arquitetura limpa de código até o rigor de formulação científica e rastreabilidade de evidências. A tabela a seguir estabelece a rastreabilidade direta entre o conteúdo formativo do programa e os itens avaliados neste edital:

Disciplina do Programa	Carga	Competência Alvo	Item Correspondente no Desafio
Dados, Experimentação e Metodologia Científica	30h	Hipóteses, desenho experimental, métricas e evidências.	Parte 3: Relatório Científico (H_0 vs H_1, gráficos, tabelas e validade).
Engenharia de Software, Requisitos e Gestão	24h	Requisitos funcionais, ágil, rastreabilidade e Git.	Parte 1: Commits semânticos, issues, README e modularidade.
Metodologia de Projetos, Ética e Soft Skills	30h	PBL, ética profissional, proteção de dados e transparência.	Declaração de uso de IA, licenças, conformidade e clareza documental.
Arquitetura de Software	18h	Clean Architecture, SOLID, desacoplamento e testabilidade.	Parte 1: Camadas `domain`, `data`, `presentation` e DIP.
Inteligência Artificial: Fundamentos e Frameworks	36h	Modelos, métricas, overfitting e inferência on-device.	Parte 1 (Conceitos) e **Trilha 1** (Pipeline de treino e TFLite).
Programação Java/Kotlin e Android	42h	Kotlin idiomático, Coroutines, Flow, Room e Compose.	Parte 1: App funcional, observabilidade reativa e banco de dados.
Testes, Automação, Segurança e CI/CD	30h	Testes unitários, automação, GitHub Actions e LGPD.	Parte 1: Suíte JUnit/MockK, pipeline CI e **Trilha 2** (Segurança).
Trilhas Especializadas (Módulos 2, 3 ou 4)	75h	Aprofundamento em IA, Cibersegurança ou Eficiência.	Parte 2: Escolha de 1 trilha aplicada correspondente à vaga pretendida.

2. Visão Geral do Desafio: Projeto "Android Sentinel Core"

Dispositivos móveis modernos operam em ambientes complexos onde múltiplas aplicações concorrem por tempo de CPU, ciclos de GPU/NPU, memória RAM e orçamento energético restrito. O candidato deverá desenvolver o **Android Sentinel Core**, uma solução enxuta que monitora a integridade do dispositivo, detecta anomalias de execução e otimiza o uso de recursos.



O desafio está estruturado em três blocos objetivos de avaliação:

- **Parte 1 – Núcleo Comum Obrigatório (50 Pontos):** Engenharia de software, arquitetura limpa em Kotlin, app Android reativo com persistência e pipeline de testes/CI.
- **Parte 2 – Módulo Especializado da Trilha Escolhida (30 Pontos):** O candidato seleciona e implementa **apenas uma** das três trilhas (Trilha 1: IA; Trilha 2: Segurança; Trilha 3: Eficiência Energética).
- **Parte 3 – Relatório Técnico-Científico e Rastreabilidade (20 Pontos):** Documento experimental obrigatório em formato de artigo técnico com formulação de hipótese e dados empíricos.

3. Especificação da Parte 1: Módulo Obrigatório (Núcleo Comum)

Todos os candidatos devem implementar a aplicação base em Kotlin (Android API 26+), estruturada com foco em manutenibilidade, testabilidade e baixo consumo de recursos computacionais.

3.1 Arquitetura e Engenharia de Software (15 Pontos)

- **Clean Architecture:** O projeto deve respeitar a separação formal em três camadas:
  • `domain`: Entidades de negócio (ex.: `TelemetrySnapshot`) e Casos de Uso (ex.: `MonitorSystemHealthUseCase`, `RecordAnomalyUseCase`) totalmente independentes do SDK Android.
  • `data`: Implementação de repositórios, Data Access Objects (Room DAO) e provedores de leitura de hardware/sensores.
  • `presentation`: ViewModels gerenciando estados de UI através de `StateFlow` ou `SharedFlow`, conectados a uma interface declarativa em **Jetpack Compose** (ou Views tradicionais com ViewBinding).
- **Inversão de Dependências (DIP):** A camada de apresentação e os casos de uso devem depender exclusivamente de interfaces/abstrações, não de implementações concretas (utilize Hilt, Koin ou injeção manual limpa).
- **Concorrência Estruturada:** Utilização idiomática de **Kotlin Coroutines** com despacho correto de threads (`Dispatchers.IO` para I/O e banco, `Dispatchers.Default` para processamento e `Dispatchers.Main` para UI).

3.2 Coleta de Telemetria e Persistência Local (15 Pontos)

- **Mecanismo de Coleta em Tempo Real:** Implementar rotina assíncrona periódica (ex.: a cada 5 ou 10 segundos) que capture:
  • *Uso de CPU:* Carga percentual instantânea (global ou por processo) via `/proc/stat`, `ActivityManager` ou dados do Mock Dataset.
  • *Memória RAM:* Memória utilizada (MB), memória livre (MB) e flag indicativa de pressão de memória (`isLowMemory`).
  • *Bateria & Térmico:* Nível da bateria (%), temperatura (°C), voltagem (mV) e status de carregamento via `BatteryManager`.
  • *Rede:* Tipo de conexão (Wi-Fi, Dados Móveis) e contadores de tráfego (bytes RX e TX).
- **Persistência Local:** Armazenamento local com **Room Database** ou **DataStore**, permitindo consultas históricas ordenadas por timestamp e retenção dos últimos 500 registros.
- **Modo Dataset Sintético (Offline Mock):** O sistema **deve** conter uma flag ou seletor de fonte de dados permitindo carregar o arquivo `starter_kit/mock_telemetry_dataset.json` emulando a telemetria, assegurando que o app seja 100% testável em qualquer máquina de avaliação.

3.3 Testes Automatizados e Pipeline de CI/CD (20 Pontos)

- **Testes Unitários:** Escrever suíte de testes com **JUnit** e **MockK** cobrindo os UseCases de domínio e repositórios. Devem ser validadas regras de negócio como detecção de limites críticos (ex.: alerta quando CPU > 85% ou temperatura > 40°C) e fluxo correto de exceções.
- **Pipeline de Integração Contínua:** Fornecer um arquivo `.github/workflows/ci.yml` funcional que execute a cada `push` ou `pull_request`:
  1. Análise estática de código (ktlint, detekt ou android lint).
  2. Compilação do projeto (`./gradlew assembleDebug`).
  3. Execução automatizada dos testes unitários (`./gradlew testDebugUnitTest`) com publicação do relatório de testes.

> **Contrato de Código Fornecido**
> Consulte o arquivo `starter_kit/ArchitectureSample.kt` no repositório para inspecionar os contratos recomendados para as entidades de telemetria, Casos de Uso e interfaces das três trilhas especializadas.

4. Especificação da Parte 2: Módulos Especializados (Escolha 1 Trilha)

O candidato deve optar por **uma única trilha**. Não é necessário implementar as três; a avaliação valoriza profundidade técnica, rigor experimental e qualidade da entrega no escopo selecionado.

Trilha 1: Inteligência Artificial aplicada à Otimização de Sistemas Android

ESPECIALIZAÇÃO IA (30 PTS)

Contexto: O AOSP gera fluxos contínuos de métricas. Identificar comportamentos degenerativos (loops infinitos, vazamentos progressivos de memória e estrangulamento térmico) antes do encerramento forçado do processo exige modelos preditivos ultraleves executados localmente no dispositivo (on-device).

Entregáveis Objetivos da Trilha:

1. **Notebook / Script de Treinamento:** Utilizando o script auxiliar `starter_kit/baseline_evaluator.py` e o dataset fornecido, desenvolva em Python um script ou notebook realizando a normalização dos atributos e o treinamento de um modelo para **Detecção de Anomalias** ou **Classificação de Estado**.
2. **Estudo Comparativo:** Compare tecnicamente ao menos duas técnicas (ex.: *Isolation Forest* vs *Regressão Logística* ou *Random Forest*). Apresente métricas formais: Matriz de Confusão, Acurácia, Precisão, Recall e F1-Score.
3. **Otimização On-Device & Quantização:** Exporte o modelo final para o formato **TensorFlow Lite (`.tflite`)** ou **ONNX Runtime Mobile**. Analise a redução no tamanho do artefato e a estimativa de latência de inferência no dispositivo.
4. **Integração na Aplicação:** Implemente na camada `data/domain` do Android Sentinel a classe consumidora do modelo (conforme contrato `OnDeviceAnomalyClassifier`), emitindo alertas preditivos na interface quando a probabilidade de anomalia superar o limiar estipulado.

Trilha 2: Segurança de Dispositivos Móveis e Automação de Validações

ESPECIALIZAÇÃO SEGURANÇA (30 PTS)

Contexto: A arquitetura de segurança do Android apoia-se em sandboxes, permissões declaradas e isolamento de processos (SELinux). Configurações equivocadas no `AndroidManifest.xml` ou comunicação insegura via Binder/IPC abrem brechas para elevação de privilégios e vazamento de dados.

Entregáveis Objetivos da Trilha:

1. **Modelagem de Ameaças (STRIDE & OWASP MASVS):** Elabore uma matriz detalhada de modelagem de ameaças para o aplicativo utilizando as categorias do **STRIDE** (Spoofing, Tampering, Repudiation, Information Disclosure, DoS, Elevation of Privilege) e correlacionando-as aos requisitos do **OWASP MASVS v2.0** (MASVS-STORAGE, MASVS-CRYPTO, MASVS-PLATFORM, MASVS-NETWORK).
2. **Auditoria de Código & Análise de Vulnerabilidades:** Identifique, documente e apresente código corrigido (hardening) para ao menos **quatro vulnerabilidades clássicas** no ecossistema Android:
   • Componente IPC exposto sem permissão (`android:exported="true"`).
   • Armazenamento inseguro de dados sensíveis (SharedPreferences em texto claro vs `EncryptedSharedPreferences`).
   • Falta de validação de canal TLS / Ausência de Certificate Pinning.
   • Emissão de dados sensíveis ou tokens em logs do sistema (Logcat em builds de release).
3. **Script Automatizado de Verificação de Segurança:** Desenvolva um script (em Python, Bash ou teste automatizado Android) que inspecione de forma determinística o APK ou dispositivo via ADB, verificando flags de compilação (`android:allowBackup`, `android:debuggable`, permissões concedidas) e emitindo um relatório automatizado com status de conformidade (Pass/Fail).

Trilha 3: Eficiência Energética avaliada por Algoritmos de Otimização

ESPECIALIZAÇÃO ENERGIA (30 PTS)

Contexto: A sincronização de dados e o processamento em background respondem pela maior fatia do consumo indesejado de bateria no Android. WakeLocks não liberados e frequências excessivas de polling impedem os núcleos de CPU de entrarem em estados de baixo consumo (Doze Mode / C-States).

Entregáveis Objetivos da Trilha:

1. **Protocolo Experimental de Medição Energética:** Formule um protocolo consistente para mensurar o impacto do envio de telemetria sob diferentes cenários de uso (sincronização síncrona a cada 5s vs envio em lotes a cada 5 minutos; Wi-Fi vs Dados Móveis 5G; tela ligada vs Doze Mode).
2. **Modelagem Matemática do Custo Energético:** Formule analiticamente uma função objetivo multiobjetivo que quantifique o trade-off entre energia, atraso de entrega e risco de perda de telemetria:
   `Minimizar: Custo(P) = w1 × E(Energia_mA) + w2 × L(Latencia_s) + w3 × D(Perda_%)`
   Sujeito a: restrições de nível crítico de bateria (< 15%) e estado de repouso do sistema.
3. **Algoritmo de Otimização Heurística:** Implemente em Kotlin ou Python um algoritmo de busca heurística (ex.: *Algoritmo Genético*, *Simulated Annealing* ou *Heurística Gulosa Adaptativa*) que selecione dinamicamente a melhor janela de transmissão e tamanho de lote com base nas restrições energéticas atuais.
4. **Validação e Redução Comprovada:** Demonstre numericamente, utilizando as medições do `baseline_evaluator.py` ou dados via `dumpsys batterystats`, o ganho percentual de eficiência energética da abordagem otimizada em comparação à abordagem ingênua (unbuffered polling).

5. Especificação da Parte 3: Relatório Técnico-Científico (20 Pontos)

O relatório técnico-científico consolida o perfil investigativo exigido pelo programa. O candidato deve preencher e entregar o documento no formato do template oficial fornecido em `starter_kit/TEMPLATE_RELATORIO_TECNICO.md` (2 a 4 páginas de extensão).

Roteiro Mandatório do Relatório Técnico

1. **Resumo Executivo (Abstract):** Síntese de 100 a 150 palavras cobrindo motivação, hipótese, metodologia, dados quantitativos e conclusão.
2. **Formulação de Hipótese:**
   • *Hipótese Nula (H_0):* A intervenção proposta (algoritmo de otimização / modelo de IA / correção de segurança) não altera significativamente o comportamento ou a eficiência do sistema móvel.
   • *Hipótese Alternativa (H_1):* A intervenção produz melhora estatisticamente observável ($\Delta \ge 25\%$) em consumo, latência ou resiliência.
3. **Desenho Experimental:** Definição explícita das variáveis independentes (políticas/configurações testadas), variáveis dependentes (métricas de medição) e variáveis de controle (dispositivo, versão de SO, temperatura ambiente).
4. **Resultados e Evidências Empíricas:** Tabela comparativa e gráficos demonstrando o comportamento do sistema antes e depois da intervenção (apoiando-se nos valores extraídos pelo `baseline_evaluator.py`).
5. **Ameaças à Validade & Limitações:** Identificação honesta de limitações do teste (ruído em emuladores, generalização entre fabricantes de SoC, variações térmicas externas).
6. **Rastreabilidade de Código:** Apontamento direto dos hashes de commit e arquivos que suportam as conclusões apresentadas.

6. Matriz e Rubrica Detalhada de Avaliação (0 a 100 Pontos)

A banca examinadora adotará critérios objetivos e pontuações padronizadas conforme a matriz abaixo:

Critério Avaliado	Peso	Indicadores de Nível Excelente (Nota Máxima)	Penalidades & Não Conformidades
1. Arquitetura & Qualidade de Código	**20 pts**	Separação estrita em camadas (Domain/Data/Presentation); princípios SOLID aplicados com naturalidade; injeção de dependência funcional; tratamento não bloqueante com Coroutines/Flow.	Lógica de negócio acoplada na View/Activity; dependência de classes Android no Domain; código não formatado (-10 pts).
2. Implementação Android Core	**20 pts**	Coleta funcional de telemetria (CPU, RAM, bateria, rede); persistência com Room/DataStore; UI declarativa reativa (Compose); alternador para dados sintéticos do Mock Dataset.	App sofre crash em execução; ausência de persistência local; interface gráfica congela por bloqueio da Main Thread (-12 pts).
3. Testes Automatizados & CI	**15 pts**	Testes unitários cobrindo casos felizes e cenários de borda dos UseCases e Repositórios; workflow do GitHub Actions rodando lint, build e testes com sucesso.	Ausência de testes unitários (-10 pts); pipeline de CI ausente ou falhando (-5 pts).
4. Desafio da Trilha Especializada	**25 pts**	Entrega integral dos requisitos da trilha escolhida (IA: modelo treinado, métricas e on-device; Sec: STRIDE, MASVS e script; Energia: modelo multiobjetivo e algoritmo heurístico).	Trilha não implementada ou com código meramente ilustrativo sem comprovação prática (-15 a -25 pts).
5. Relatório Técnico & Rastreabilidade	**20 pts**	Hipótese clara (H_0 vs H_1); dados quantitativos com gráficos e tabelas; discussão de ameaças à validade; histórico Git semântico com commits rastreáveis.	Relatório vago sem evidências numéricas (-10 pts); commit único "first commit" demonstrando falta de processo (-8 pts).

7. A Base de Apoio para o Candidato ("Starter Kit" e Recursos)

Para eliminar barreiras de infraestrutura e garantir produtividade imediata, este repositório disponibiliza uma base completa de artefatos pré-configurados:

7.1 O Dataset Sintético de Telemetria (`starter_kit/mock_telemetry_dataset.json`)

O arquivo JSON fornecido contém 120 leituras temporais amostradas a cada 10 segundos (20 minutos de histórico), cobrindo cinco regimes de estresse do sistema operacional Android:

- **Amostras 0 a 29 (Baseline Idle):** Funcionamento em repouso com CPU estável entre 4% e 14% e corrente de -150 mA.
- **Amostras 30 a 49 (CPU Spike & Aquecimento Térmico):** Carga de CPU sustentada acima de 85% e temperatura elevando-se até 42°C (*cenário ideal para modelos preditivos da Trilha 1*).
- **Amostras 50 a 69 (Wakelock Leak & Tráfego de Rádio):** WakeLock permanente impedindo o sono do dispositivo e tráfego massivo com corrente em -920 mA (*cenário ideal para otimização da Trilha 3*).
- **Amostras 70 a 89 (Inundação IPC & Anomalia de Segurança):** Tentativas repetidas de acesso a componentes internos exportados sem assinatura (*cenário ideal para validações da Trilha 2*).
- **Amostras 90 a 119 (Doze Mode & Deep Sleep):** Recuperação do sistema com transição para sono profundo e corrente contida em -65 mA.

7.2 Script Auxiliar de Baseline (`starter_kit/baseline_evaluator.py`)

Execute `python starter_kit/baseline_evaluator.py` para obter instantaneamente:

```
$ python starter_kit/baseline_evaluator.py
================================================================
 ANDROID SENTINEL CORE - RELATÓRIO DA LINHA DE BASE (BASELINE)
================================================================
 Amostras analisadas:      120 leituras | Período: 20.0 minutos
 Uso Médio de CPU:         32.19% (Pico: 99.5%)
 Temperatura Média:        32.75 °C (Pico: 41.5 °C)
 Corrente Média:           399.29 mA | Descarga Acumulada: 133.1 mAh
 Anomalias Registradas:    60 (50.0%)
================================================================
```

7.3 Diretrizes Éticas e Uso de Inteligência Artificial

> **Declaração de Transparência e Honestidade Intelectual**
> O uso de ferramentas de assistência baseadas em IA (Copilot, ChatGPT, Claude, Gemini) é **permitido como ferramenta de suporte e produtividade**. É **obrigatório** declarar no `README.md` quais ferramentas foram utilizadas e em quais etapas. Na etapa de arguição técnica, o candidato será avaliado individualmente quanto ao domínio integral da solução entregue.

8. Instruções de Submissão e Checklist Final

1. **Repositório:** Submeter o link de um repositório no GitHub ou GitLab contendo todo o código-fonte, branches e histórico de desenvolvimento.
2. **Commits:** Histórico de commits atômicos e descritivos seguindo a convenção *Conventional Commits* (`feat:` , `fix:` , `test:` , `docs:` , `refactor:`).
3. **Documentação (README.md):** Deve conter instruções claras de compilação, versão da JVM recomendada (Java 17), comandos de teste e a identificação da trilha escolhida.
4. **Relatório Técnico:** O relatório final (`RELATORIO_TECNICO.md` ou versão em PDF) deve estar comitado na raiz do repositório, com todas as seções preenchidas.

9. Bibliografia Canônica e Fontes Oficiais Recomendadas

Recomenda-se a fundamentação nas seguintes fontes oficiais consagradas na indústria e na academia:

Tema	Referência Canônica	Endereço Eletrônico Oficial
Arquitetura Android	Guide to App Architecture & Jetpack Modern Practices	`https://developer.android.com/topic/architecture`
AOSP Internals	Android Open Source Project Architecture & Core Services	`https://source.android.com/docs/core`
Cibersegurança Móvel	OWASP Mobile Application Security Verification Standard (MASVS v2.0)	`https://mas.owasp.org/MASVS/`
Profiling de Desempenho	Perfetto: Modern System Profiling and Trace Analysis	`https://perfetto.dev/docs/`
Eficiência Energética	Android Power Management, Battery Historian & Doze Mode	`https://developer.android.com/topic/performance/power`
IA On-Device	TensorFlow Lite & Android Machine Learning Solutions	`https://www.tensorflow.org/lite/android`
Qualidade de Software	Martin, Robert C. *Clean Architecture: A Craftsman's Guide*	Prentice Hall / Pearson Education

---

Coordenação Técnica do Programa de Residência Tecnológica em Sistemas Android Avançados
Documento oficial de seleção técnica e nivelamento acadêmico • Uso Institucional e de Candidatos • Ciclo 2026/2027